SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 30%

QUESTIONS: 2

Duration : 45 Minutes
Total Marks:20

Annexure A

Descriptive Written Test

Code of Conduct:

I Ashwanth SP/192126002 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



ASSESSMENT TOOL 3 – DRY RUN EVALUATION

COURSE NAME: ARTIFICIAL INTELLIGENCE

COURSE CODE: CSA17 | COURSE OUTCOME: CO2

Submitted by: Vishal Balaji J | Register Number: 192525014

CO2 focuses on applying search techniques and game-solving methods to solve problems systematically. This dry run demonstrates how Breadth-First Search (BFS), Depth-First Search (DFS), and BFS for the 8-puzzle explore states and maintain their data structures.

1. Breadth-First Search (BFS)

Breadth-First Search explores a graph level by level. It uses a FIFO (first-in, first-out) queue: the node that enters the queue first is removed and expanded first. Starting from A, the algorithm visits A, then its children B and C, followed by the next level from left to right. The following trace assumes that children are considered from left to right.

Dry-run table

Iteration	Queue after removing current node and adding its children	Visited node
1	[B, C]	A
2	[C, D, E]	B
3	[D, E, F, G]	C
4	[E, F, G]	D
5	[F, G]	E
6	[G]	F
7	[]	G

BFS traversal order: A → B → C → D → E → F → G.

The queue initially contains [A]. At each iteration, the front element is removed and marked visited. Its unvisited children are added to the rear of the queue. After A is expanded, B and C are waiting. Expanding B adds D and E behind C; expanding C adds F and G behind D and E. This queue order is the reason BFS completes one level before moving to the next.

BFS is complete for a finite graph when repeated states are managed correctly. In an unweighted graph, it also finds a path with the fewest edges. Its main limitation is memory usage, because the queue can become large when many nodes exist at the same level.

2. Depth-First Search (DFS)

Depth-First Search explores one branch as far as possible before returning to an earlier node. It can be implemented with a stack or recursion. For this dry run, the graph is explored from left to right, and the stack top is shown at the right-hand side. When using an explicit stack, children are pushed in reverse order so the leftmost child is visited first.

Dry-run table

Iteration	Stack (top at right) after expansion	Visited node
1	[C, B]	A
2	[C, E, D]	B
3	[C, E]	D
4	[C]	E
5	[G, F]	C
6	[G]	F
7	[]	G

DFS traversal order: A → B → D → E → C → F → G.

The initial stack contains A. After A is visited, its children are arranged so B is processed before C. DFS follows B to D, then backtracks to E, and returns to C only after B's branch is complete. It then visits F and G. This trace illustrates the central feature of DFS: it follows a branch deeply before exploring another branch.

DFS often uses less memory than BFS because it stores the current path and pending alternatives rather than an entire frontier. However, it does not generally guarantee the shortest path. In graphs with cycles, a visited set or path-checking mechanism is needed to avoid repeatedly visiting the same nodes.

3. 8-Puzzle Problem Using BFS

The 8-puzzle consists of eight numbered tiles and one blank space on a 3 × 3 board. A legal move slides a tile next to the blank into that space. BFS treats each board configuration as a state and each legal slide as an edge. Since every move has the same cost, BFS can find a solution using the minimum number of moves, provided that repeated states are not added to the search again.

Initial and goal states

Initial state	Goal state
1 3 6 5 0 2 4 7 8	1 2 3 4 5 6 7 8 0

The blank is represented by 0. The search begins with the initial state in the queue. At each step, BFS removes the oldest state, generates all legal blank moves, and appends any previously unseen states. The move order used for the trace below is Up, Down, Left, then Right; a different move-generation order can change the order of states explored, but not the shortest solution length.

Shortest solution path

Step	Board state	Move made
Initial	1 3 6	—

	5 0 2 4 7 8	
1	1 3 6 5 2 0 4 7 8	Right
2	1 3 0 5 2 6 4 7 8	Up
3	1 0 3 5 2 6 4 7 8	Left
4	1 2 3 5 0 6 4 7 8	Down
5	1 2 3 0 5 6 4 7 8	Left
6	1 2 3 4 5 6 0 7 8	Down
7	1 2 3 4 5 6 7 0 8	Right
8 (Goal)	1 2 3 4 5 6 7 8 0	Right

The solution requires 8 moves: Right → Up → Left → Down → Left → Down → Right → Right. Each move shifts the blank in the stated direction. The final board matches the goal state exactly.

Answers to the questions

1. Which node is expanded in each iteration? In the BFS tree-search process, the node at the front of the queue is expanded first. For the 8-puzzle, the expanded nodes are board configurations rather than single letter labels. The exact order depends on the legal-move ordering and whether duplicate states are excluded.

2. How many states are generated in total? The assignment does not specify the order for generating legal moves or whether ‘generated’ means all successor states or only unique states added to the queue. Therefore, a single count cannot be stated independently of the implementation. In a correct implementation, count each newly discovered unique configuration when it is first added to the queue, and report that count together with the move order and stopping rule.

3. Write the complete solution path. The path is shown in the table above and reaches the goal in eight moves.

4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem? BFS expands all states at depth 0, then depth 1, then depth 2, and so on. Every move has equal cost, so the first time BFS reaches the goal, no solution with fewer moves can remain unexplored. This guarantee relies on tracking visited states to prevent redundant cycles.

Conclusion

The dry runs demonstrate that the choice of data structure determines the order in which search states are explored. BFS uses a queue and explores level by level, making it suitable when a shortest path is required in an unweighted problem. DFS uses a stack and explores deeply before backtracking, which can reduce memory use but does not guarantee the shortest path. In the 8-puzzle, BFS systematically examines board configurations and returns a shortest sequence of eight moves when the stated move-generation order and duplicate-state handling are used.